



main.c

 Share

Run

Output

Clear

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4
5 int main()
6 {
7     pid_t pid;
8
9     pid = fork();
10
11     if(pid < 0)
12     {
13         printf("Process creation failed.\n");
14     }
15     else if(pid == 0)
16     {
17         printf("\n----- Child Process ----- \n");
18         printf("Child Process ID (PID) : %d\n", getpid());
19         printf("Parent Process ID (PPID): %d\n", getppid());
```

```
----- Parent Process -----
Parent Process ID (PID): 8266
Parent Process ID (PPID): 8259

----- Child Process -----
Child Process ID (PID) : 8267
Parent Process ID (PPID): 8266

=== Code Execution Successful ===
```

main.c

```
1 #include <stdio.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <string.h>
5
6 int main()
7 {
8     int shmid;
9     char *str;
10
11     // Create shared memory
12     shmid = shmget((key_t)1234, 1024, 0666 | IPC_CREAT);
13
14     if(shmid == -1)
15     {
16         printf("Shared memory creation failed.\n");
17         return 1;
18     }
19
```

Output

Data written into shared memory:  
Hello! This is Shared Memory IPC.

=== Code Execution Successful ===

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <sys/ipc.h>
4 #include <sys/msg.h>
5
6 struct message
7 {
8     long msg_type;
9     char msg_text[100];
10 };
11
12 int main()
13 {
14     struct message msg;
15
16     // Create a message queue
17     int msgid = msgget((key_t)1234, 0666 | IPC_CREAT);
18
19     if(msgid == -1)
```

```
Enter a message: welcome to os lab
Message sent successfully.

Received Message: welcome to os lab

=== Code Execution Successful ===
```

main.c

```
46 {
47     tat[i] = wt[i] + bt[i];
48     avgwt += wt[i];
49     avgtat += tat[i];
50 }
51
52 printf("\nProcess\tBT\tWT\tTAT\n");
53
54 for(i = 0; i < n; i++)
55 {
56     printf("P%d\t%d\t%d\t%d\n",
57         p[i], bt[i], wt[i], tat[i]);
58 }
59
60 printf("\nAverage Waiting Time = %.2f", avgwt / n);
61 printf("\nAverage Turnaround Time = %.2f\n", avgtat / n);
62
63 return 0;
64 }
```

Output

```
Enter number of processes: 4
Enter Burst Time for P1: 6
Enter Burst Time for P2: 8
Enter Burst Time for P3: 7
Enter Burst Time for P4: 3

Process BT WT TAT
P4 3 0 3
P1 6 3 9
P3 7 9 16
P2 8 16 24

Average Waiting Time = 7.00
Average Turnaround Time = 13.00

=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i, tq;
6     int bt[20], rt[20], wt[20], tat[20];
7     int time = 0, done;
8     float avgwt = 0, avgtat = 0;
9
10    printf("Enter number of processes: ");
11    scanf("%d", &n);
12
13    for(i = 0; i < n; i++)
14    {
15        printf("Enter Burst Time for P%d: ", i + 1);
16        scanf("%d", &bt[i]);
17
18        rt[i] = bt[i];
19        wt[i] = 0;
```

```
Enter number of processes: 4
Enter Burst Time for P1: 5
Enter Burst Time for P2: 4
Enter Burst Time for P3: 2
Enter Burst Time for P4: 1
Enter Time Quantum: 2
```

| Process | BT | WT | TAT |
|---------|----|----|-----|
| P1      | 5  | 7  | 12  |
| P2      | 4  | 7  | 11  |
| P3      | 2  | 4  | 6   |
| P4      | 1  | 6  | 7   |

```
Average Waiting Time = 6.00
Average Turnaround Time = 9.00
```

```
=== Code Execution Successful ===
```

main.c

```
50 printf("\nProcess\tPriority\tBT\tCT\tWT\tTAT\n");
51
52 for(i = 0; i < n; i++)
53 {
54     tat[i] = ct[i];
55     wt[i] = tat[i] - bt[i];
56
57     avgwt += wt[i];
58     avgtat += tat[i];
59
60     printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n",
61           i + 1, pr[i], bt[i], ct[i], wt[i], tat[i]);
62 }
63
64 printf("\nAverage Waiting Time = %.2f", avgwt / n);
65 printf("\nAverage Turnaround Time = %.2f\n", avgtat / n);
66
67 return 0;
68 }
```

Output

Clear

```
Enter Priority for P1: 8
Enter Burst Time for P2: 8
Enter Priority for P2: 1

Enter Burst Time for P3: 7
Enter Priority for P3: 4

Enter Burst Time for P4: 3
Enter Priority for P4: 2

Process Priority    BT  CT  WT  TAT
P1 8        6  24  18  24
P2 1        8   8   0   8
P3 4        7  18  11  18
P4 2        3  11   8  11

Average Waiting Time = 9.25
Average Turnaround Time = 15.25
```

main.c

```
4 - {  
5     int n, i, j, temp;  
6     int bt[20], pr[20], p[20];  
7     int wt[20], tat[20];  
8     float avgwt = 0, avgtat = 0;  
9  
10    printf("Enter the number of processes: ");  
11    scanf("%d", &n);  
12  
13    for(i = 0; i < n; i++)  
14    {  
15        p[i] = i + 1;  
16  
17        printf("\nEnter Burst Time for P%d: ", p[i]);  
18        scanf("%d", &bt[i]);  
19  
20        printf("Enter Priority for P%d: ", p[i]);  
21        scanf("%d", &pr[i]);  
22    }
```

Output

Enter Priority for P1: 3  
Enter Burst Time for P2: 8  
Enter Priority for P2: 1  
Enter Burst Time for P3: 7  
Enter Priority for P3: 4  
Enter Burst Time for P4: 3  
Enter Priority for P4: 2  

| Process | Priority | BT | WT | TAT |
|---------|----------|----|----|-----|
| P2      | 1        | 8  | 0  | 8   |
| P4      | 2        | 3  | 8  | 11  |
| P1      | 3        | 6  | 11 | 17  |
| P3      | 4        | 7  | 17 | 24  |

Average Waiting Time = 9.00  
Average Turnaround Time = 15.00







